# Backend · Integración con Stockfish

Oct 12, 2026 · @German

El modo vs. Stockfish funciona: el backend abre Stockfish como proceso hijo, le habla por protocolo UCI y responde una jugada en unos 1-2 segundos. Se probó con Stockfish 17.1 en Windows (encontró un mate en 1 y contestó 1.e4 con c5). No se modificó nada del trabajo de Desarrollo Simulación. Backend escribió solo el adaptador mínimo que necesitaba para poder probar el modo vs. Stockfish, con lo que ya estaba definido en Arquitectura (sección 6.1). Según el Manual de Roles, el motor es responsabilidad de Simulación: ellos deciden si adoptan este adaptador, lo cambian o lo reemplazan por su MotorStockfish.

## Cómo funciona

1. El humano juega con `POST /games/:id/moves`. La respuesta trae `machineTurn: true`.
2. El frontend llama a `POST /games/:id/engine-move`.
3. GestorDePartida reconstruye el tablero desde las jugadas guardadas y le pide la jugada al oponente que tiene el turno (patrón Strategy, GameSessionFactory).
4. StockfishOpponent traduce la dificultad a Skill Level (Fácil 3, Medio 10, Difícil 20) y llama a StockfishAdapter.
5. StockfishAdapter le manda a Stockfish por stdin: `setoption name Skill Level value X`, `isready`, `position fen <FEN>`, `go movetime 1000`, y lee de stdout la línea `bestmove e7e5`.
6. Tablero valida que la jugada sea legal, y la jugada y el nuevo FEN se guardan en una sola transacción.
7. La respuesta trae la jugada de Stockfish y el estado nuevo de la partida.

Detalles de diseño:

- Un solo proceso de Stockfish, compartido por todas las partidas y reutilizado entre jugadas. Se abre en la primera jugada de la máquina.
- Los pedidos se atienden de a uno (cola), porque UCI no admite dos búsquedas a la vez.
- Al apagar el servidor se cierra el proceso, para que no quede colgado en Windows.
- Código: `backend/src/engines/StockfishAdapter.js` y `StockfishOpponent.js`.

## Configuración y prueba

1. Bajar Stockfish de stockfishchess.org y descomprimirlo.

2. En `backend/.env` poner la ruta al ejecutable, por ejemplo `STOCKFISH_PATH=C:\Users\yo\Escritorio\stockfish\stockfish-windows-x86-64-universal.exe`.
3. Levantar el servidor con `pnpm dev:memory`. La consola muestra la línea `Stockfish:` con la ruta que va a usar.

| Dificultad | Skill Level | Tiempo de búsqueda | ELO informado |
|---|---|---|---|
| Fácil | 3 | 1000 ms | 800-1200 |
| Medio | 10 | 1000 ms | 1300-1700 |
| Difícil | 20 | 1000 ms | 1800+ |

Los valores salen de la tabla difficulty_profiles: si el profesor decide otra cosa, se cambian ahí, no en el código.

Pruebas: `pnpm test` corre 10 pruebas del adaptador con un motor UCI falso, así que funcionan en cualquier PC. Con `STOCKFISH_TEST_PATH` apuntando al ejecutable se suma una prueba contra Stockfish real.

## Errores y qué hace el frontend

| HTTP | code | Cuándo | Qué hacer en pantalla |
|---|---|---|---|
| 504 | ENGINE_TIMEOUT | Stockfish no respondió en 3000 ms (STOCKFISH_TIMEOUT_MS) | Ofrecer "Reintentar" o "Pausar" (CU-03 flujo 3a). El tablero no cambió. |
| 503 | ENGINE_UNAVAILABLE | La ruta está mal, el programa no arranca o se cerró | Mostrar el mensaje y ofrecer reintentar. |
| 502 | ENGINE_BAD_MOVE | Stockfish devolvió una jugada ilegal (no debería pasar) | Reintentar. No se guardó nada. |
| 409 | NOT_YOUR_TURN | Se llamó a engine-move en el turno del humano | No llamar a engine-move salvo con machineTurn: true. |

Si Stockfish se cuelga, el backend cierra ese proceso y abre uno nuevo en el próximo pedido. Esto resuelve el defecto que QA describió en su reporte de ejemplo PF-03.

# Pendiente

- [ ] Simulación (Paula, Lázaro, Luciano): decidir si usan este adaptador o el suyo. Si usan el suyo, solo tiene que respetar getBestMove(fen, opciones) y Backend no cambia nada más.
- [ ] Arquitectura (Thomas): la sección 6.1 coincide con lo implementado; actualizar la sección 8 (endpoints y puerto 3001).
- [ ] GC (Facundo): documentar en la guía de entorno dónde se instala Stockfish y la variable STOCKFISH_PATH.
- [ ] Profesor: confirmar si la dificultad va por Skill Level o por ELO (UCI_LimitStrength + UCI_Elo).